

python-KernSmooth

A Technical Report on Porting the
KernSmooth R Package to Python

Yufei Cai

`/groups/jli9/Yufei/python-KernSmooth`

Status: Work in Progress

Last updated: 2026-05-18

Contents

Abstract	3
1 Introduction	3
1.1 Background and Motivation	3
1.2 Repository Structure	4
1.3 Automated Workflow Infrastructure	4
2 Phase 1: Static Analysis of the KernSmooth R Package	5
2.1 Motivation	5
2.2 R Dependency Analysis	5
2.3 Architecture Synthesis	6
2.4 Key Findings	6
2.4.1 Package Scope and External Dependencies	6
2.4.2 Function Inventory	7
2.4.3 Three-Level Dependency Hierarchy	7
2.4.4 Fortran Integration Convention	8
2.4.5 Architectural Design: Binned Approximation	8
2.4.6 Note on Fortran Source Analysis	8
3 Phase 2: Establishing the Python Build Infrastructure	8
3.1 Motivation	8
3.2 Build System Design	9
3.2.1 Fortran Sources	9
3.2.2 f2py Wrapper Generation	10
3.2.3 NumPy Include Directories	10
3.2.4 BLAS Discovery and Fallback Chain	10
3.2.5 Package Structure	11
3.3 Installation	11
4 Phase 3: Language Dependency Catalogue and Conversion Guides	12
4.1 Motivation	12
4.2 Language Dependency Extraction	12
4.3 Conversion Guide Generation	13
4.4 Selected Translation Nuances	14
4.4.1 FFT Normalisation (<code>fft.md</code>)	14
4.4.2 Fortran FFI Type Coercions (<code>as.double.md</code> , <code>as.integer.md</code>)	14
4.4.3 Sample Variance Denominator (<code>var.md</code>)	14
4.4.4 Optional Argument Detection (<code>missing.md</code>)	14
4.4.5 Negative-Base Fractional Exponentiation (<code>sqrt.md</code>)	14
4.4.6 Array Memory Order (<code>matrix.md</code>)	15
4.4.7 Package Lifecycle Hooks (<code>library.dynam.unload.md</code>)	15
4.4.8 Partial String Matching (<code>match.arg.md</code>)	15
4.4.9 R switch on Strings (<code>switch.md</code>)	15
5 Phase 4: Automated Function Conversion and Package Assembly	15

5.1	Motivation	15
5.2	Function Conversion	16
5.2.1	Notable Per-Function Translation Decisions	16
5.3	Package Assembly	17
5.4	The f2py Return-Value Bug	17
5.4.1	Observed Symptom	17
5.4.2	Root Cause: R .Fortran() vs. f2py Semantics	18
5.4.3	Fortran Output Argument Audit	18
5.4.4	Patches Applied	18
5.5	Output Validation	19
6	Phase 5: Code Quality Audit and Namespace Refinement	19
6.1	Motivation	19
6.2	Type Annotation Completion	20
6.3	Namespace Privacy Investigation	20
6.4	Full Code Audit and Systematic Refactoring	21
6.4.1	Dead Code from R Porting	21
6.4.2	Silent Crash Bug in Three Public Functions	21
6.4.3	Structural Code Duplication	21
6.4.4	PEP 8 Signature Reformatting	22
6.5	State of the Package After Phase 5	22
	Appendices	24
A	Agent Specifications	24
A.1	Agents Used in Phase 1	24
	analyze-r-file-dependencies	24
A.2	Agents Used in Phase 3	25
	extract-r-file-language-dependencies	25
	generate-language-dependency-conversion-guide	26
A.3	Agents Used in Phase 4	27
	convert-r-function-to-python	27
A.4	Supplementary Agent	28
	analyze-c-file-dependencies	28
B	Skill (Command) Specifications	30
B.1	Skills Used in Phase 1	30
	analyze-r-folder-dependencies	30
B.2	Skills Used in Phase 3	30
	extract-r-folder-language-dependencies	30
	generate-language-dependency-conversion-guides	31
B.3	Skills Used in Phase 4	32
	convert-r-file-to-python	32
	combine-python-functions-into-file	33
B.4	Workflow Utility	34
	summarize-research-report	34

Abstract

This report documents the ongoing work to port the R package `KernSmooth` (version 2.23-26, released 2024-12-10) into an installable Python package of the same name. `KernSmooth` is a *recommended* package shipped with every standard R installation; it implements kernel smoothing methods from Wand & Jones [1] and has no runtime dependencies on any external CRAN package.

The conversion strategy retains the original Fortran 77 computational core verbatim, compiles it into a Python C-extension via `numpy.f2py`, and replaces the R wrapper layer with semantically equivalent Python functions. The project proceeds in five phases: (1) static architectural analysis of the R package, (2) Python build-system establishment, (3) language-dependency cataloguing and per-dependency conversion guide generation, (4) automated R-to-Python function conversion and package assembly, and (5) code-quality audit and namespace refinement. All five phases have been executed. The resulting package installs under Python 3.14 (CPython) in a `conda` environment on a Red Hat Enterprise Linux 9 HPC node and produces numerically correct output.

1 Introduction

1.1 Background and Motivation

The R package `KernSmooth` implements the statistical methods described in

M. P. Wand and M. C. Jones (1995). *Kernel Smoothing*. Chapman and Hall, London.

It is a *recommended* package — one of the roughly fifteen packages that ship as part of the base R distribution and are maintained by the R Core Team. The package was authored by M. P. Wand, contributed LINPACK Fortran routines are attributed to Cleve Moler, and the current R-language port and maintenance is by Brian Ripley. It is released under an “Unlimited” license.

The package provides seven public functions covering kernel density estimation, plugin bandwidth and bin-width selection, and local polynomial regression. Because it delegates all numerically intensive loops to compiled Fortran 77 code, it is both correct and performant.

The motivation for a Python port is to make these well-tested implementations available to Python-based scientific workflows without requiring an R runtime. A port that re-uses the *original Fortran routines* offers a stronger correctness guarantee than a ground-up Python reimplement: numerical differences between the Python and R packages can then originate only in the R-to-Python wrapper translation, which is far easier to audit systematically.

The broader scientific motivation, experimental design, and validation plan for this project are described in two planning documents that are not reproduced in this report:

- `docs/methodology_and_plan_Jun.pdf` — the primary methodology and project

plan.

- docs/methodology_addendum_20260511.pdf — an addendum dated 2026-05-11.

The original R package reference manual is archived at docs/KernSmooth.pdf. The contents of these PDF files were not available during the writing of this report and are therefore not summarised here.

1.2 Repository Structure

The working directory python-KernSmooth/ is organized as follows.

Path	Contents
KernSmooth/ py-KernSmooth/	Unmodified R package source (v2.23-26). Python package under development; contains KernSmooth/__init__.py, meson.build, pyproject.toml, Fortran sources in src/, and tests/test.py.
structural_analysis/	Dependency-graph visualisation artefacts (dependency_graph.{py,html,pdf,png}), the function-level CSV (dependency_levels.csv), and the JSON analysis outputs per R file.
language_dependency_analysis/	Per-file CSV dependency tables, the CSV merge script, the combined table, and the 46 per-dependency conversion guides in conversion_guides/.
conversion_results/	Per-function JSON translation artefacts produced by the conversion pipeline.
docs/	Architecture document, planning PDFs, per-phase summaries in daily_summaries/, and this report in report/.
.claude/agents/	Sub-agent specification Markdown files.
.claude/commands/	Skill (slash command) specification Markdown files.

1.3 Automated Workflow Infrastructure

This project employs **Claude Code** (Anthropic) as an AI-assisted development environment. Recurring multi-step tasks are encoded as *skills* — top-level slash commands whose specifications are stored as Markdown files under .claude/commands/. Skills orchestrate one or more *sub-agents* whose specifications are stored under .claude/agents/. Both types of specification files use YAML front matter to declare a name and description, followed by a structured natural-language body describing execution steps and output schemas. Sub-agents are dispatched via the @agent-name notation from within a skill.

The full specifications for all agents and commands referenced in this report appear in Appendices [A](#) and [B](#).

At the end of each phase, the `/summarize-research-report` skill was invoked:

```
/summarize-research-report
```

This produced a structured Markdown summary of the phase’s activities, which was saved to `docs/daily_summaries/`. Those summaries are the primary source material for this report.

2 Phase 1: Static Analysis of the KernSmooth R Package

2.1 Motivation

Before any translation work could begin, a precise, machine-readable map of the R package’s internal structure was required. The goals of this phase were:

1. Classify every R function by dependency type: calls to R standard-library functions, calls to other functions defined within the same (or other self-written related) R source file, and calls through the Fortran foreign-function interface.
2. Establish the public API surface and the complete internal call graph.
3. Understand the Fortran integration layer and the symbol-registration convention it uses.
4. Produce a comprehensive architecture document to serve as a reference throughout all subsequent phases.

Without this foundation, later phases would need to infer architectural facts on a per-function basis, risking inconsistency and missed dependencies. The structural analysis JSON produced here also serves as a direct input to the language-dependency extraction skill in Phase 3 and to the conversion ordering logic in Phase 4.

2.2 R Dependency Analysis

The `/analyze-r-folder-dependencies` skill (Appendix [B](#), §[B.1](#)) was invoked against the R source directory:

```
/analyze-r-folder-dependencies KernSmooth/R/
```

The skill performs a recursive scan of the target folder for `.R` and `.r` files, then dispatches the `@analyze-r-file-dependencies` sub-agent (Appendix [A](#), §[A.1](#)) on each file. Because `KernSmooth` packages all R code into a single source file (`KernSmooth/R/all.R`), exactly one agent invocation was required.

The sub-agent parses the R file, identifies every user-defined function, and classifies each function call in its body into one of three mutually exclusive categories:

language_dependencies Calls to R built-in functions or functions from installed packages, identified by namespace prefix or by not being locally defined (e.g. `fft`, `var`, `dnorm`, `stats::quantile`).

internal_dependencies Calls to other functions defined within the same R file.

external_dependencies Calls via `.Fortran()`, `.C()`, `.Call()`, or `.External()` to compiled native-code entry points.

The result is a JSON object keyed by function name, with each value containing the three dependency arrays, each listing unique function names for the dependencies. The output was written to `structural_analysis/R/all.json`.

2.3 Architecture Synthesis

To produce a comprehensive architecture document, the following files were read in parallel and synthesised into `docs/architecture-analysis.md`:

File	Purpose
<code>KernSmooth/DESCRIPTION</code>	Package metadata, authorship, license, declared dependencies.
<code>KernSmooth/NAMESPACE</code>	Exported symbols; <code>useDynLib</code> registration directive.
<code>structural_analysis/R/all.json</code>	Per-function dependency classification (produced above).
<code>structural_analysis/dependency_level</code>	Function call-depth hierarchy (pre-existing artefact).
<code>KernSmooth/man/bkde.Rd</code>	Density estimation API and algorithm description.
<code>KernSmooth/man/bkde2D.Rd</code>	Two-dimensional density estimation API.
<code>KernSmooth/man/bkfe.Rd</code>	Kernel functional estimator API.
<code>KernSmooth/man/dpih.Rd</code>	Histogram bin-width selector API.
<code>KernSmooth/man/dpik.Rd</code>	Kernel density bandwidth selector API.
<code>KernSmooth/man/dpill.Rd</code>	Local linear regression bandwidth selector API.
<code>KernSmooth/man/locpoly.Rd</code>	Local polynomial estimator API.

A check for `KernSmooth/vignettes/` confirmed that the directory does not exist; the package has no vignette sources.

2.4 Key Findings

2.4.1 Package Scope and External Dependencies

`KernSmooth` has *zero* runtime dependencies on contributed CRAN packages. It imports exactly five functions from the base `stats` package: `dbeta`, `dnorm`, `fft`, `quantile`, and `var`. `MASS` and `carData` appear in the `Suggests` field of `DESCRIPTION` but are

referenced only in `man/` examples and play no role in the package’s computational logic. This minimal dependency footprint simplifies the Python port considerably.

2.4.2 Function Inventory

`R/all.R` defines exactly 16 functions in a single file. Seven are exported via `NAMESPACE`; nine are internal (unexported).

Function	Exported	Level	Fortran entry point
<code>bkde</code>	Yes	0	<code>F_linbin</code> (via <code>linbin</code>)
<code>bkde2D</code>	Yes	0	<code>F_lbtwod</code> (via <code>linbin2D</code>)
<code>dpih</code>	Yes	0	<code>F_linbin</code> (via <code>linbin</code>)
<code>dpik</code>	Yes	0	<code>F_linbin</code> (via <code>linbin</code>)
<code>dpill</code>	Yes	0	multiple (via six helpers)
<code>locpoly</code>	Yes	0	<code>F_locpol</code> and <code>F_rlbin</code>
<code>bkfe</code>	Yes	1	<code>F_linbin</code> (via <code>linbin</code>)
<code>linbin</code>	No	2	<code>F_linbin</code>
<code>rlbin</code>	No	2	<code>F_rlbin</code>
<code>blkest</code>	No	1	<code>F_blkest</code>
<code>cpblock</code>	No	1	<code>F_cp</code>
<code>linbin2D</code>	No	1	<code>F_lbtwod</code>
<code>sdiag</code>	No	1	<code>F_sdiag</code>
<code>sstdiag</code>	No	1	<code>F_sstdg</code>
<code>.onAttach</code>	No	0	—
<code>.onUnload</code>	No	0	—

`bkfe` is notable for being both an *exported* function and an *internal dependency*: `dpih` and `dpik` both call it to evaluate the density functionals $\int f(x) f^{(r)}(x) dx$ required for plug-in bandwidth selection.

2.4.3 Three-Level Dependency Hierarchy

The internal call graph has three levels, stored in `structural_analysis/dependency_levels.csv`:

Level 0 (entry points): Functions callable directly by the user: `bkde`, `bkde2D`, `dpih`, `dpik`, `dpill`, `.onAttach`, `.onUnload`.

Level 1 (mid-tier helpers): Functions called by one or more level-0 functions: `bkfe`, `blkest`, `cpblock`, `linbin2D`, `locpoly`, `sdiag`, `sstdiag`.

Level 2 (binning primitives): `linbin` and `rlbin`.

`linbin` (one-dimensional linear binning, backed by Fortran `F_linbin`) is the single most central function: it is called by seven distinct callers (`bkde`, `bkfe`, `dpih`, `dpik`, `locpoly`, `sdiag`, `sstdiag`), meaning that every density estimation and functional estimation path passes through it.

`dpill` is the most internally complex exported function, orchestrating six internal helpers (`rlbin`, `cpblock`, `blkest`, `locpoly`, `sdiag`, `sstdiag`) to produce a single bandwidth estimate.

2.4.4 Fortran Integration Convention

The NAMESPACE file contains:

```
useDynLib(KernSmooth, .registration = TRUE, .fixes = "F_")
```

This registers all Fortran entry points at package load time and applies the `F_` prefix to their R-level names, preventing symbol conflicts. All eight Fortran-delegating R wrappers use this naming scheme. LINPACK-derived linear-algebra routines (attributed to Cleve Moler in DESCRIPTION) reside in `KernSmooth/src/d*` and underpin the local polynomial computations in `locpoly`, `sdiag`, and `sstdiag`.

2.4.5 Architectural Design: Binned Approximation

Every computationally non-trivial function in `KernSmooth` adopts the same two-phase strategy: (1) **bin** the raw data onto an equally-spaced grid using *linear binning*, which preserves the first moment of each data point within its bin interval (unlike simple histogram binning), and (2) **convolve** the bin counts with a kernel weight vector, either via FFT (`bkde`, `bkde2D`, `bkfe`) or via the Fortran local-polynomial routines. This trades a small, controllable bias for a reduction in computational complexity from $O(n \cdot g)$ to $O(n + g \log g)$, where n is the sample size and g is the grid size.

2.4.6 Note on Fortran Source Analysis

A complete cross-language dependency map would also require analysing the compiled Fortran sources in `KernSmooth/src/`. The `/analyze-c-folder-dependencies` skill (Appendix B) was identified as a candidate tool for this task (it handles both C and Fortran), but this analysis has not yet been executed. Due to the nature of `KernSmooth` package, the Fortran sources are relatively self-contained and do not call back into R, so the lack of a Fortran-level dependency map has not posed a significant obstacle to subsequent phases. The Fortran code is treated as a black box that is called by the R wrappers, so the focus has been on understanding the R-level structure and dependencies.

3 Phase 2: Establishing the Python Build Infrastructure

3.1 Motivation

The conversion strategy requires the original Fortran 77 sources to be compiled into a Python C-extension so that Python wrapper functions can call them directly. This approach avoids re-implementing numerically sensitive routines and provides a strong guarantee of numerical equivalence with the R package.

The R package's Fortran integration uses R's internal `.Fortran()` mechanism. Replicating this in Python requires `f2py` (part of NumPy), which parses Fortran source files and automatically generates C wrapper code and Python callable interfaces. The

resulting extension must be integrated into a proper Python package with an installable structure.

Phase 2 addresses all build-system and packaging concerns before any wrapper code is written.

3.2 Build System Design

The build system uses `meson-python` as the build backend, declared in `py-KernSmooth/pyproject.toml`.

```
[build-system]
build-backend = "mesonpy"
requires = ["meson-python", "numpy"]

[project]
name = "KernSmooth"
version = "0.1.0"
dependencies = ["numpy"]
```

The complete build definition resides in `py-KernSmooth/meson.build`.

3.2.1 Fortran Sources

Thirteen Fortran 77 fixed-form source files in `py-KernSmooth/src/` are compiled into the extension:

File	Origin
<code>src/blkest.f</code>	Original <code>KernSmooth</code> source
<code>src/cp.f</code>	Original <code>KernSmooth</code> source
<code>src/dgedi.f</code>	Original <code>KernSmooth</code> source (LINPACK LU)
<code>src/dgefa.f</code>	Original <code>KernSmooth</code> source (LINPACK LU)
<code>src/dgesl.f</code>	Original <code>KernSmooth</code> source (LINPACK LU)
<code>src/linbin.f</code>	Original <code>KernSmooth</code> source
<code>src/linbin2D.f</code>	Original <code>KernSmooth</code> source
<code>src/locpoly.f</code>	Original <code>KernSmooth</code> source
<code>src/rlbin.f</code>	Original <code>KernSmooth</code> source
<code>src/sdiag.f</code>	Original <code>KernSmooth</code> source
<code>src/sstdiag.f</code>	Original <code>KernSmooth</code> source
<code>src/dqrdc.f</code>	Added from Netlib (LINPACK QR decomposition, G. W. Stewart 1978)
<code>src/dqrsl.f</code>	Added from Netlib (LINPACK QR solve, G. W. Stewart 1978)

The two Netlib additions are necessary because `blkest.f` and `cp.f` call `dqrdc_` and `dqrsl_`, respectively. The original R package satisfies these symbols from R's own internal LINPACK copy (built into the R binary). OpenBLAS provides only BLAS levels 1–3 and does not include LINPACK QR routines. Sourcing them from Netlib is the standard remedy for this gap when porting R packages that depend on LINPACK.

The presence of LINPACK routines (both the original `dgedi/dgefa/dgesl` trio and the newly added `dqrdc/dqrs1`) was discovered iteratively during Phase 2: each link failure at `pip install` time revealed one missing symbol, which was then resolved by adding the corresponding Netlib file.

3.2.2 f2py Wrapper Generation

`f2py` (NumPy 2.4.3) is invoked as a Meson `custom_target`:

```
python -m numpy.f2py @INPUT@ -m _KernSmooth --lower
```

Because all sources are Fortran 77 fixed-form (`.f`), `f2py` generates `_KernSmoothmodule.c` and `_KernSmooth-f2pywrappers.f` (the fixed-form wrapper variant). An initial build template incorrectly named the wrapper file `_KernSmooth-f2pywrappers2.f90` (the free-form Fortran 90 variant), which caused a build failure; the distinction was identified by inspecting the `f2py` output and corrected in the `meson.build` outputs list.

The `-lower` flag lowercases all Fortran symbol names in the generated C wrapper, which is required for standard GFortran symbol conventions.

3.2.3 NumPy Include Directories

The Meson configure step must locate NumPy’s C header files at configure time (not at install time), since they are required to compile `fortranobject.c`, the `f2py` runtime support file. Two `run_command` calls retrieve these directories:

```
incdir_numpy = run_command(py,
    ['-c', 'import numpy; print(numpy.get_include())'],
    check: true).stdout().strip()
incdir_f2py = run_command(py,
    ['-c', 'import numpy.f2py; print(numpy.f2py.get_include())'],
    check: true).stdout().strip()
```

`fortranobject.c` (located in `incdir_f2py`) is included directly in the extension module’s source list.

3.2.4 BLAS Discovery and Fallback Chain

The HPC node (Red Hat Enterprise Linux 9, GFortran 15.2.0, `ld.bfd 2.35.2-67`) provides OpenBLAS at `/usr/lib64/libopenblas.so.0`, but the unversioned symlink (`libopenblas.so`) normally installed by the `openblas-openmp-devel` package is absent. Meson’s standard `find_library` and `dependency('blas')` mechanisms cannot resolve a bare `.so.0` file. The following four-stage fallback chain in `meson.build` addresses this:

```
blas_dep = dependency('blas', required: false)
if not blas_dep.found()
    blas_dep = fc.find_library('openblas', required: false)
endif
if not blas_dep.found()
```

```

    blas_dep = fc.find_library('openblas', required: false)
endif
if not blas_dep.found()
    blas_dep = fc.find_library('blas', required: false)
endif
if not blas_dep.found()
    blas_dep = declare_dependency(link_args:
        ['/usr/lib64/libopenblas.so.0'])
endif

```

The final `declare_dependency` hard-codes the absolute path as a last resort. The presence of each fallback stage allows the build to succeed on other systems that may have BLAS installed in the standard way.

3.2.5 Package Structure

The Python package exposes two components:

KernSmooth/_KernSmooth The compiled Fortran C-extension (private backend). All 11 original subroutines are accessible via `KernSmooth._KernSmooth.name`.

KernSmooth/__init__.py The Python wrapper layer that defines the public API.

An early design exposed the raw Fortran subroutines directly in the `KernSmooth` namespace. This was identified as architecturally incorrect: the Fortran subroutines have a low-level calling convention (all arguments passed by reference, no input validation, no Python-idiomatic defaults) and should not be part of the public API. `__init__.py` was revised to contain only `from . import _KernSmooth` and `__all__ = []`, deferring all public-API decisions to the wrapper functions to be written in Phase 4.

3.3 Installation

The package is installed into the `r-to-python` conda environment via:

```
pip install --no-build-isolation .
```

The `-no-build-isolation` flag is required because the conda environment provides `numpy` and `meson-python` directly, and the Meson configure step calls `numpy.get_include()` and `numpy.f2py.get_include()` at configure time. With build isolation, pip would install `numpy` into an isolated build environment, producing a different path that may conflict with the runtime environment.

The full invocation, including the cluster's `module load gcc` and `conda activate r-to-python` steps, is captured in `py-KernSmooth/pip_install.sh`. This script is also suitable for submission as a cluster batch job (`#$ -pe smp 1, #$ -q long` directives are included).

After successful installation, `import KernSmooth` and `import KernSmooth._KernSmooth` both succeed without error.

4 Phase 3: Language Dependency Catalogue and Conversion Guides

4.1 Motivation

The automated function conversion in Phase 4 maps each R construct to its Python equivalent. For most R standard-library functions, a naïve translation to the apparent NumPy or SciPy analogue introduces silent numerical errors. Documented examples include: R’s `fft(inverse=TRUE)` is unnormalised while NumPy’s `ifft` normalises by $1/N$; R’s `var()` uses Bessel’s correction while `np.var` does not; R’s `as.integer()` targets 32-bit integers while NumPy defaults to 64-bit.

One may argue that these discrepancies are well-known and can be handled on a case-by-case basis during the conversion process in Phase 4. However, due to the random nature of the LLM, we cannot guarantee that all language dependencies will be correctly identified and translated whenever a function is converted. If there is not a unified, machine-readable reference of all language dependencies and their translation nuances, then the risk of silent numerical errors in the final Python package is unacceptably high. (Even if the translation is mostly correct, we cannot ensure the consistency of choices or styles, which can make debugging significantly difficult.) Also, there are many language dependencies in R that have so many variations in their usage that it is hard to guarantee a correct translation without a comprehensive reference. For example, the `as.double()` function is used in many different contexts (such as on scalars, vectors, matrices, or even data frames), and its translation to Python can vary depending on the context. If we rely directly on the LLM to identify and translate each usage correctly during the conversion process, we may miss some cases or introduce errors. By contrast, if we systematically extract all language dependencies and their call sites in Phase 3, and then generate detailed conversion guides for each scenario for each dependency, we can ensure that the translation in Phase 4 is consistent and accurate across all functions.

Phase 3 systematically catalogs every R standard-library function used in `all.R`, records its precise call sites, and generates a dedicated translation guide for each. By encoding these nuances in machine-readable Markdown files that are later fed to the conversion sub-agent, the translation quality is improved without depending on the agent’s training-data knowledge of R semantics solely.

4.2 Language Dependency Extraction

The `/extract-r-folder-language-dependencies` skill (Appendix B, §B.2) was invoked:

```
/extract-r-folder-language-dependencies \
  KernSmooth/R/ \
  structural_analysis/R/
```

The skill dispatches the `@extract-r-file-language-dependencies` sub-agent (Appendix A, §A.2) on each R file together with its pre-computed structural-analysis JSON.

The sub-agent reads the structural JSON to identify the language dependencies of each function, then rescans the R source line by line to record, for every individual call site: the dependency name, the enclosing function name, the line number, and the complete call expression (including multi-line calls).

The result was written to `language_dependency_analysis/R/all.csv`: a 370-row, 4-column CSV (`language_dependency`, `function_name`, `line_number`, `call_body`) covering all 16 functions in `all.R`.

A utility script `language_dependency_analysis/combine_csvs.py` was also written to:

- Recursively glob all `.csv` files under `language_dependency_analysis/R/`.
- Insert a `file_path` column (the relative path of the source R file within the `R/` folder, `.csv` extension replaced with `.R`).
- Concatenate, sort by [`language_dependency`, `file_path`, `function_name`, `line_number`], and save the combined result to `language_dependency_analysis/combined_table.csv`.

This script is designed to sort the entries in the original `all.csv` to make the entries for the same language dependency contiguous and convenient to be extracted for further tasks. It also makes it easier to support future analysis of additional R files beyond `all.R`.

4.3 Conversion Guide Generation

The `/generate-language-dependency-conversion-guides` skill (Appendix B, §B.2) was invoked:

```
/generate-language-dependency-conversion-guides \  
  KernSmooth/R/ \  
  language_dependency_analysis/combined_table.csv
```

The skill identified 46 unique `language_dependency` values in the combined CSV and launched 46 instances of the `@generate-language-dependency-conversion-guide` sub-agent (Appendix A, §A.2) in a single parallel batch. Each agent:

1. Received the CSV subset for its assigned dependency.
2. Read the corresponding source lines in `all.R` for contextual understanding of argument types and surrounding logic.
3. Consulted R online documentation (<https://www.rdocumentation.org>) where the function's behavior was complex or non-obvious.
4. Produced a structured Markdown guide specifying the correct Python translation strategy, with concrete code examples.

The 46 resulting guides were saved to `language_dependency_analysis/conversion_guides/` with filenames matching the dependency name (e.g. `fft.md`, `var.md`, `match.arg.md`).

4.4 Selected Translation Nuances

The following are the most consequential findings from the conversion guides, as they directly informed specific decisions in Phase 4. Together, they constitute the principal risk of silent numerical error in any R-to-Python translation of this package.

4.4.1 FFT Normalisation (`fft.md`)

R's `fft(..., inverse=TRUE)` returns the *raw, unnormalised* inverse DFT: for an N -point transform it returns $\sum_{k=0}^{N-1} X_k e^{2\pi i k n / N}$ without dividing by N . NumPy's `np.fft.ifft` normalises by $1/N$ by default, as per the conventional definition. The R source explicitly divides the inverse-FFT result by the grid size P (or $P_1 \times P_2$ for two-dimensional transforms) after computing `Re()`. When porting to NumPy, those division operations must be *removed*.

4.4.2 Fortran FFI Type Coercions (`as.double.md`, `as.integer.md`)

R's pre-allocation idiom `as.double(rep(0, n))` maps to `np.zeros(n, dtype=np.float64)`. `as.integer(Lvec)` on a vector maps to `Lvec.astype(np.int32)`. The 32-bit integer constraint is non-obvious but critical: standard Fortran `INTEGER` is 32 bits; passing `np.int64` arrays would silently corrupt the values seen by the Fortran subroutine on a 64-bit system.

4.4.3 Sample Variance Denominator (`var.md`)

R's `var(x)` uses Bessel's correction ($n - 1$ denominator), i.e. it computes the *sample* variance. NumPy's `np.var` defaults to the *population* variance (n denominator, `ddof=0`). All occurrences of `sqrt(var(x))` in the R source must be translated as `np.std(x, ddof=1)`.

4.4.4 Optional Argument Detection (`missing.md`)

R's `missing(param)` returns `TRUE` if the caller did not supply the argument. Python has no built-in equivalent. The idiomatic replacement is the `None` sentinel default: `def f(x, param=None)` with `if param is None:` guards inside the function body.

4.4.5 Negative-Base Fractional Exponentiation (`sqrt.md`)

Several bandwidth formulas involve odd roots of quantities that may be negative, e.g. $(-3\sqrt{2/\pi} / \hat{\psi}_6)^{1/7}$. R silently computes real-valued odd roots of negative numbers via its internal rule for fractional exponents. Python's `**` operator raises `ValueError` for negative bases with non-integer exponents. The correct idiom is

```
math.copysign(abs(x) ** (1.0 / n), x)
```

4.4.6 Array Memory Order (`matrix.md`)

R's `matrix()` and Fortran both use *column-major* (Fortran-order) storage. NumPy defaults to *row-major* (C-order). When reshaping arrays produced by Fortran subroutines, `np.reshape(..., order='F')` is required.

4.4.7 Package Lifecycle Hooks (`library.dynam.unload.md`)

R provides package attach/detach lifecycle hooks via `.onAttach` and `.onUnload`. `library.dynam.unload` has no Python equivalent: CPython does not safely support unloading native extension modules. The guide documents that the `.onUnload` body should either be omitted or replaced with an `atexit` callback for non-library cleanup tasks.

4.4.8 Partial String Matching (`match.arg.md`)

R's `match.arg(arg, choices)` validates `arg` against a vector of allowed strings and supports *unambiguous prefix matching* (e.g. "ep" matches "epanechnikov"). No standard Python equivalent exists. The guide specifies a reusable helper function that performs exact match, then unique-prefix match, then raises a descriptive error.

4.4.9 R `switch` on Strings (`switch.md`)

R's `switch(key, case1=expr1, case2=expr2, ...)` on string keys translates cleanly to a Python dictionary lookup. For cheap scalar computations (as in `KernSmooth`), a plain dict of values is sufficient; a dict of lambda functions is available for cases requiring lazy evaluation.

5 Phase 4: Automated Function Conversion and Package Assembly

5.1 Motivation

With the dependency map (Phase 1), build infrastructure (Phase 2), and translation guides (Phase 3) in place, Phase 4 executes the actual conversion. Two challenges make this non-trivial even with the preparation work done:

1. Functions must be converted in dependency order: when the conversion agent for a function reads the JSON files of its callees to understand their Python calling conventions, those files must already exist.
2. The Fortran FFI calling convention in Python (via `f2py`) differs fundamentally from R's `.Fortran()` in how it handles output arguments. This difference is invisible from the R source and therefore not naturally handled by pattern matching; it requires targeted post-conversion patching.

5.2 Function Conversion

The `/convert-r-file-to-python` skill (Appendix B, §B.3) was invoked:

```
/convert-r-file-to-python \
  KernSmooth/R/all.R \
  structural_analysis/R/all.json \
  structural_analysis/dependency_levels.csv \
  language_dependency_analysis/conversion_guides/ \
  conversion_results/R/
```

The skill parses `dependency_levels.csv`, filters for `r_file == all.R`, and derives a topological ordering from leaves to roots. It then invokes the `@convert-r-function-to-python` sub-agent (Appendix A, §A.3) once per function, *sequentially* in topological order, providing each agent with:

- The R source fragment for the function.
- The function’s JSON dependency map (from `all.json`).
- The folder of 46 language-dependency conversion guides.
- The output folder `conversion_results/R/all.R/` for the resulting JSON artefact.

The conversion order was:

Level	Functions (in conversion order)
2 (deepest leaves)	<code>linbin</code> , <code>rlbin</code>
1 (intermediate)	<code>bkfe</code> , <code>blkest</code> , <code>cpblock</code> , <code>linbin2D</code> , <code>locpoly</code> , <code>sdiag</code> , <code>sstdiag</code>
0 (roots)	<code>.onAttach</code> , <code>.onUnload</code> , <code>bkde</code> , <code>bkde2D</code> , <code>dpik</code> , <code>dpik</code> , <code>dpik</code>

The 16 resulting JSON artefacts were written to `conversion_results/R/all.R/`. (Two artefacts — `.onAttach.json` and `.onUnload.json` — use leading-dot filenames and are not visible to standard `ls`; they require `ls -a` to discover.)

5.2.1 Notable Per-Function Translation Decisions

linbin/rlbin/linbin2D/blkest/cpblock `.Fortran(F_xxx, ...)[[k]]` mapped to `_KernSmooth.xxx(...)` with pre-allocated NumPy output buffers.

bkfe Hermite polynomial recurrence loop translated directly; the manual `/P` division present in the R source was removed per the `fft.md` guide, since NumPy’s `np.fft.ifft` already normalises.

locpoly Dual density/regression branch (controlled by whether the `y` argument is `None`); a variable-length bandwidth discretization loop; a 19-argument Fortran subroutine call; and a `math.gamma(drv+1)` post-multiplier for the derivative-order correction.

sdiag/sstdiag The `ss`, `Smat`, `uu`, `Umat` matrices are flattened column-major (`order='F'`) before being passed to the Fortran subroutine, consistent with the `matrix.md` guide.

dpik/dpik Negative-base fractional exponents (e.g. $(-3\sqrt{2/\pi/\hat{\psi}_6})^{1/7}$) replaced with `math.copysign(abs(val)**(1/k), val)` per the `sqrt.md` guide.

bkde2D Two-dimensional FFT convolution via `np.fft.fft2/np.fft.ifft2`; wrap-around kernel construction using Python slice notation to replicate R’s decreasing-index sequences.

.onAttach packageStartupMessage translated to `print(..., file=sys.stderr)`.

.onUnload library.dynam.unload body omitted per `library.dynam.unload.md`; function registered as an `atexit` callback at module level.

5.3 Package Assembly

The `/combine-python-functions-into-file` skill (Appendix B, §B.3) was invoked:

```
/combine-python-functions-into-file \
  conversion_results/R/all.R/ \
  py-KernSmooth/KernSmooth/__init__.py
```

Before writing, the skill performed a structural scan of `meson.build` and the existing `__init__.py` to verify the correct import path for the Fortran extension (`from . import _KernSmooth`), then corrected discrepancies found in the per-function JSON artefacts:

Original (in JSON artefacts)	Corrected
<code>from KernSmooth import _KernSmooth</code>	<code>from . import _KernSmooth</code>
<code>from KernSmooth.linbin import linbin</code>	removed (functions are co-located)
<code>from scipy.stats import norm (duplicate)</code>	merged: <code>from scipy.stats import beta as beta_dist, norm</code>

The final import block, sorted standard library → third-party → local, was:

```
import atexit
import math
import sys
import warnings

import numpy as np
from scipy.stats import beta as beta_dist, norm

from . import _KernSmooth
```

The `__all__` list was populated with all 14 public function names (excluding `_on_attach` and `_on_unload`). Functions were written to the file in leaf-to-root order, ensuring all callees are defined before callers. Syntax was confirmed immediately by `ast.parse()`.

5.4 The f2py Return-Value Bug

5.4.1 Observed Symptom

After assembly, running the test script raised:

```

TypeError: 'NoneType' object is not subscriptable
File "__init__.py", line 36, in linbin
    gcnts = _KernSmooth.linbin(...)[6]

```

5.4.2 Root Cause: R `.Fortran()` vs. `f2py` Semantics

R's `.Fortran()` always returns a *named list of all arguments*, including modified output arrays. The R idiom `.Fortran(F_linbin, ...)$gcnts` (or equivalently `[[6]]`) extracts the modified output array from that list.

`f2py`-wrapped Fortran *subroutines* return `None` unconditionally. Output array arguments are modified in-place via C pointer; the Python caller reads results by examining the pre-allocated arrays that were passed in. Scalar output arguments require special treatment: passing a Python `np.float64(0.0)` (immutable) does not allow Fortran to write back. The fix is to use a length-1 NumPy array (`np.zeros(1, dtype=np.float64)`), which is mutable and passes a writable pointer.

The `@convert-r-function-to-python` agent for `linbin` had generated `[6]` subscripting on the `None` return. The agent for `rlbin` (converted first) had correctly identified the in-place semantics and avoided this. The inconsistency arose because each agent operated independently.

5.4.3 Fortran Output Argument Audit

All Fortran source files were read to identify output arguments:

Subroutine	Output argument(s)	Type
<code>linbin</code>	<code>gcnts(*)</code>	double precision array
<code>lbtwod</code>	<code>gcnts(*)</code>	double precision array
<code>blkest</code>	<code>sigsqe, th22e, th24e</code>	double precision <i>scalars</i>
<code>cp</code>	<code>Cpvals(Nmax)</code>	double precision array
<code>locpol</code>	<code>cvest(*)</code>	double precision array
<code>sdiag</code>	<code>Sdg(*)</code>	double precision array
<code>sstdg</code>	<code>SSTd(*)</code>	double precision array

5.4.4 Patches Applied

Seven targeted edits were made to `py-KernSmooth/KernSmooth/__init__.py`:

1. **`linbin`** — Removed `[6]` subscript; read `gcnts` (pre-allocated `np.zeros(M)`) directly after the call.
2. **`linbin2D`** — Removed `out = ...` and `out[8]`; read `gcnts` then reshaped with `order='F'`.
3. **`blkest`** — Changed `sigsqe/th22e/th24e` from `np.float64(0.0)` to `np.zeros(1, dtype=np.float64)`; return values read as `sigsqe[0]`, `th22e[0]`, `th24e[0]`.
4. **`cpblock`** — Removed `out = ...` and `out[12]`; used `Cpvals` (modified in-place) directly in `np.argmin`.

5. **locpoly** — Removed `out = ...` and `out[18]`; applied `math.gamma(drv+1)` multiplier to `curvest` directly.
6. **sdiag** — Removed `out = ...` and `out[16]`; returned `Sdg` directly.
7. **sstdiag** — Removed `out = ...` and `out[18]`; returned `SSTd` directly.

A post-patch `grep` confirmed zero remaining stale `out[integer]` subscripts on Fortran call returns. `ast.parse()` re-confirmed syntax validity.

5.5 Output Validation

The test script `py-KernSmooth/tests/test.py` invoked `KernSmooth.bkde(np.array([1,2,3,4,5]))` with default parameters and printed the result. The output was verified analytically:

- **Grid range:** $[-4.244, 10.244]$. Expected $[\min(x) - \tau h, \max(x) + \tau h]$ where $\tau = 4$ and $h \approx \delta_0 \cdot (243/175)^{1/5} \cdot \sigma \approx 0.776 \times 1.069 \times 1.581 \approx 1.312$, giving $[-4.248, 10.248]$. ✓
- **Grid size:** 401 points (default `gridsize=401`). ✓
- **Peak location:** Maximum density ≈ 0.18989 at $x = 3.0$, matching the mean and median of the symmetric input $\{1, 2, 3, 4, 5\}$. ✓
- **Symmetry:** The density array is a palindrome. ✓
- **Non-negativity:** All values > 0 . ✓
- **Normalisation:** Trapezoidal integral ≈ 1.0 . ✓

After Phase 4, the package was in a runnable state with all 16 functions converted. `__init__.py` was approximately 990 lines.

6 Phase 5: Code Quality Audit and Namespace Refinement

6.1 Motivation

The automated conversion pipeline prioritises functional correctness. Phase 5 is a manual audit and refactoring pass that addresses three categories of follow-on concerns:

1. **Type annotations:** The conversion agent was instructed to produce complete annotations, but the initial assembly contained bare container types (`np.ndarray`, `dict`) without subtype parameters.
2. **Namespace hygiene:** Module-level imports (`import numpy as np`, etc.) are visible as package attributes; best-practice conventions for managing this were investigated and applied.
3. **Code quality:** A systematic audit identified dead code left over from the R porting process, a latent crash bug, structural duplication, and PEP 8 violations, all of which were resolved.

6.2 Type Annotation Completion

All 14 public function signatures and both private function signatures were reviewed. The following annotation deficiencies were found and corrected:

- Bare `np.ndarray` parameters and return types, which lack dtype information.
- Bare `dict`, `tuple`, and `list` without element type parameters.
- Optional parameters with default `None` missing the `| None` union form (e.g. `bandwidth`, `range_x`, `y`, `degree`).

`from typing import Any` was added to the import block. Every array annotation was expanded to `np.ndarray[Any, np.dtype[np.float64]]`, reflecting the exclusive use of 64-bit floating-point arrays throughout the module. Return types of functions returning dictionaries were updated:

- `rlbin`, `locpoly`, `sdiag`, `sstdiag`, `bkde`, `bkde2D` → `dict[str, np.ndarray[Any, np.dtype[np.float64]]]`
- `blkest` → `dict[str, np.float64]` (scalar values extracted from length-1 arrays)

The `NDArray[np.float64]` alias from `numpy.typing` was evaluated and rejected in favour of the explicit `np.ndarray` base type.

6.3 Namespace Privacy Investigation

Importing `numpy` as `np` at module level exposes `KernSmooth.np` as a public package attribute, which is undesirable but difficult to prevent. Two strategies were evaluated:

Underscore aliasing All imports renamed to `_np`, `_math`, etc., with global replacement of all references throughout the file. Implemented and tested (confirmed `hasattr(KernSmooth, 'np')` returned `False`), then **reverted**. A survey of major scientific Python packages (NumPy, SciPy, pandas) confirmed that underscore-aliasing of dependency imports is not standard practice; all use conventional import names at module level.

del cleanup Applicable only to imports used *exclusively at module initialisation time*. Only `atexit` qualified: it appeared solely in the `atexit.register(_on_unload)` call. `del atexit` was added immediately after that registration. No other import could be safely deleted because all others (`math`, `np`, etc.) are referenced inside function bodies, where deletion would cause `NameError` at call time.

PEP 562 (`module.__getattr__`) was also evaluated but found inapplicable: it is invoked only for *missing* attributes and cannot hide names already present in `__dict__`. The `__all__` list was confirmed as the authoritative and conventional public-API contract.

6.4 Full Code Audit and Systematic Refactoring

A line-by-line review of the 981-line `__init__.py` identified seven defect categories. All were resolved in a single comprehensive rewrite.

6.4.1 Dead Code from R Porting

`_on_attach` and `_on_unload` were direct transliterations of R's `.onAttach` and `.onUnload` lifecycle hooks. Python's import machinery provides no equivalent of R's package attach/detach events; neither function was reachable from any Python code path. Their presence also rendered `import atexit`, `import sys`, and the subsequently added `del atexit` statement all useless. All were removed.

An R-era source-control comment inside `cpblock` (`# remove unused 'q' 2007-07-10`) was also removed; it had no meaning in a Python file and referenced an event in the R package's version history.

6.4.2 Silent Crash Bug in Three Public Functions

`locpoly`, `sdiag`, and `sstdiag` all declared `bandwidth: np.ndarray[...] | None = None` (a `None` default with a `| None` annotation added during type annotation completion) yet called `np.asarray(bandwidth, dtype=np.float64)` unconditionally. Passing `None` produced a 0-d array with value `nan`; a subsequent `len(bandwidth)` call on that 0-d array raised `TypeError: len() of unsized object`.

The annotation advertised `None` as a semantically valid input while the implementation crashed silently. In the internal call graph, the bug was unreachable because all three functions are always called from `dpill` with a concrete `bandwidth` value — but any external caller relying on the type annotation would have encountered it immediately. A guard was added at the top of each function body:

```
if bandwidth is None:
    raise ValueError("'bandwidth' must be specified")
```

6.4.3 Structural Code Duplication

Two logic blocks appeared identically across three functions each:

Bandwidth discretization (27 lines) In `locpoly`, `sdiag`, and `sstdiag`: handles the scalar-bandwidth path, the vector-of-length- M path, and an error path. Extracted into:

```
def _discretize_bandwidth(bandwidth, M, delta, Q, tau):
    ... # returns (hdisc, Lvec, indic, Q)
```

The extraction also corrected a robustness defect in the scalar-detection logic. The original code used `elif len(bandwidth) == 1`, which raises `TypeError` for 0-d NumPy arrays produced by `np.asarray` on a plain Python float or

`np.float64` scalar. The corrected guard, consistent with the pattern already used in `bkde2D`, is:

```
if bw.ndim == 0 or len(bw) == 1:
    ...
```

Prefix-match validation (8 lines) In `bkde` (for `kernel`), `dpik` (for `scalest`), and `dpik` (for both `kernel` and `scalest`). Extracted into:

```
def _resolve_choice(val, choices, param_name):
    ... # exact match -> unique prefix match -> error
```

Both helpers are private (underscore-prefixed) and absent from `__all__`, preserving the public API surface.

6.4.4 PEP 8 Signature Reformatting

All function signatures exceeding 88 characters were reformatted to multi-line style with one parameter per line. The most extreme cases (`locpoly`, `sdiag`, `sstdiag`) had reached 210+ characters on a single line prior to reformatting.

6.5 State of the Package After Phase 5

The rewritten `py-KernSmooth/KernSmooth/__init__.py` is 1,052 lines and satisfies the following properties:

- No dead code or unused imports.
- No bare `np.ndarray`, `dict`, `tuple`, or `list` annotations in any public or private function prototype.
- A clear `ValueError` (not an opaque `TypeError`) is raised when `bandwidth=None` is passed to `locpoly`, `sdiag`, or `sstdiag`.
- No duplicated bandwidth-discretization or argument-validation logic.
- Correct 0-d NumPy array handling in the scalar-bandwidth code path.
- PEP 8 line-length compliance (≤ 88 characters) across all function signatures.
- No R-era changelog comments or R-specific lifecycle hook documentation.

The runtime target is CPython 3.14 (confirmed by the presence of `__pycache__/__init__.cpython-314`). The package uses Python 3.10+ syntax (`dict[str, ...]`, `float | None` union types, lowercase generics), which is consistent with the runtime version.

References

- [1] M. P. Wand and M. C. Jones (1995). *Kernel Smoothing*. Chapman and Hall, London.
- [2] S. J. Sheather and M. C. Jones (1991). A reliable data-based bandwidth selection method for kernel density estimation. *Journal of the Royal Statistical Society, Series B*, **53**(3), 683–690.

- [3] D. Ruppert, S. J. Sheather, and M. P. Wand (1995). An effective bandwidth selector for local least squares regression. *Journal of the American Statistical Association*, **90**(432), 1257–1270.
- [4] M. P. Wand (1994). Fast computation of multivariate kernel estimators. *Journal of Computational and Graphical Statistics*, **3**(4), 433–445.

Appendices

A Agent Specifications

This appendix reproduces the complete specifications of the sub-agents deployed in this project. Each file is stored under `.claude/agents/` in the project repository. The YAML front matter names the agent and provides a one-line description; the body specifies the execution steps and output schema in structured natural language. These files are the normative definitions of agent behaviour: what the agent receives as input, how it processes it, and what it must output.

A.1 Agents Used in Phase 1

analyze-r-file-dependencies

```

---
name: analyze-r-file-dependencies
description: Analyzes an R script to extract structural dependency information function by function,
             categorizing calls into language, internal, and external dependencies.
---

# Analyze an R File for Dependencies

## Description

When provided with an R file (‘.R’), your task is to parse the file, identify every user-defined
function, and trace all function calls made within their bodies. You must categorize these
dependencies into strict groups and output the result as a structured JSON object.

## Execution Steps

### Step 1: Extract Function Declarations

Scan the target R file and identify every user-defined function declared within it (e.g., ‘my_func <-
function(...) { ... }’). Create a list of these function names.

### Step 2: Analyze Function Bodies and Trace Calls

For every function identified in Step 1, thoroughly analyze its execution body. Extract every
function call made within that body and categorize it into one of the following three mutually
exclusive lists:

* **Language Dependencies:**
  * *Definition:* Functions provided by Base R (e.g., ‘c()’, ‘lapply()’, ‘print()’) or functions
    from installed CRAN/Bioconductor packages (e.g., ‘mutate()’, ‘ggplot()’).
  * *Heuristic:* Include any function call that includes a package namespace prefix (e.g., ‘dplyr::
    select’) or any standard function that is not defined locally.
* **Internal Dependencies:**
  * *Definition:* Functions that are defined within the exact same R file, or functions that are
    explicitly defined in other ‘.R’ files within the same project directory.
* **External Dependencies:**
  * *Definition:* Foreign language interfaces (typically C, C++, or Fortran) invoked from within
    the R code.
  * *Heuristic:* Look specifically for calls to ‘.Call()’, ‘.C()’, ‘.Fortran()’, ‘.External()’, or
    ‘useDynLib()’. Extract the name of the compiled routine being invoked (usually the first string
    argument passed to these interfaces).

### Step 3: Generate JSON Output

Construct a JSON object where each key is the name of a function identified in Step 1. The value for
each key must be an object containing exactly three arrays of strings: ‘language_dependencies’,

```

```

        'internal_dependencies', and 'external_dependencies'. Deduplicate all function names within
        these arrays.

## Output Format Schema

You must output valid JSON strictly adhering to this structure. Do not include markdown code blocks
if the system expects raw JSON.

'''json
{
  "function_name_1": {
    "language_dependencies": ["c", "lapply", "dplyr::filter"],
    "internal_dependencies": ["helper_function_within_file"],
    "external_dependencies": [".Call(\"c_routine_name\")"]
  },
  "function_name_2": {
    "language_dependencies": ["print"],
    "internal_dependencies": ["function_name_1"],
    "external_dependencies": []
  }
}
'''

```

A.2 Agents Used in Phase 3

extract-r-file-language-dependencies

```

---
name: extract-r-file-language-dependencies
description: Extracts detailed location and usage data for language dependencies in an R file,
            utilizing pre-calculated structural analysis JSON.
---

# Extract Language Dependencies from an R File

## Description

When provided with an R source file (`.R`) and its corresponding structural analysis JSON (generated
by the `@analyze-r-file-dependencies` agent), your task is to locate every instance where those
language dependencies are invoked. You must extract the exact line numbers and textual calls for
each dependency, and output the results as a well-formatted, sorted CSV.

## Execution Steps

### Step 1: Ingest Inputs
Parse the provided structural analysis JSON to identify the `language_dependencies` associated with
each user-defined function (the JSON keys).

### Step 2: Locate and Extract Invocations
Scan the raw text of the `.R` file. For every function listed in the JSON, find its declaration in
the R file and scan its body for the specific `language_dependencies` listed in its JSON array.

For every individual call to these dependencies, extract the following data points:
* **`language_dependency`**: The exact string name of the dependency as listed in the JSON (e.g., `c`, `dplyr::filter`).
* **`function_name`**: The name of the user-defined function (the JSON key) where this dependency is
  currently being executed.
* **`line_number`**: The exact line number in the `.R` file where the invocation of the dependency
  begins.
* **`call_body`**: The exact code snippet of the invocation, including its arguments (e.g., `dplyr::
  filter(df, column == "value")`). If the call spans multiple lines, capture the entire complete
  statement.

### Step 3: Format and Sort Output
Compile the extracted data points into a CSV format with a header row.

```

Output Format Schema

You must output valid CSV data strictly adhering to this structure. Include the header row. Enclose ‘call_body’ strings in double quotes to prevent formatting errors from commas or line breaks within the code.

```
'''csv
language_dependency,function_name,line_number,call_body
c,my_func_1,14,"c(1, 2, 3)"
c,my_func_2,42,"c(\"A\", \"B\")"
dplyr::filter,my_func_1,15,"dplyr::filter(data, val > 0)"
lapply,my_func_1,22,"lapply(list, function(x) {
  x + 1
})"
print,my_func_2,45,"print(\"Processing complete.\")"
'''
```

generate-language-dependency-conversion-guide

```
---
name: generate-language-dependency-conversion-guide
description: Generates a conversion guide for translating a specific language dependency from R to
  Python.
---

# Generate Language Dependency Conversion Guide

## Description

When provided with a target ‘base_folder’ and a CSV text input matching the following schema:
'''csv
language_dependency,file_path,function_name,line_number,call_body
exp,all.R,locpoly,663,"exp(seq(log(hlow), log(hupp), length.out = Q))"
exp,all.R,sdiag,768,"exp(seq(log(hlow), log(hupp), length.out = Q))"
exp,all.R,sstdiag,845,"exp(seq(log(hlow), log(hupp), length.out = Q))"
'''

Your task is to generate a comprehensive conversion guide for the specified ‘language_dependency’ (e.
g., ‘exp’). The guide must include:
1. A clear explanation of the ‘language_dependency’'s core functionality in R.
2. A detailed analysis of its usage within the provided CSV data, including the surrounding context
retrieved directly from the source R files.
3. A conversion guide to equivalent Python functions/methods, providing general Python code snippets
for all scenarios in the CSV.

## Execution Steps

### Step 1: Ingest Inputs and Gather Context

- Iterate through each row in the CSV. Using the provided ‘base_folder’, navigate to the exact ‘
file_path’ and ‘line_number’.
- Read the target line and the surrounding code blocks. You must read enough lines to fully capture
multi-line function calls, identify the data types of arguments being passed (e.g., scalars vs.
vectors), and understand how the result interacts with adjacent logic.
- If the R dependency’s behavior is complex or ambiguous, search ‘https://www.rdocumentation.org/’ to
confirm its official functionality, default arguments, and edge cases.
- If there is ambiguity about the valid types of the functions’ parameters or return values, you can
also look them up in the document ‘https://cran.r-project.org/web/packages/KernSmooth/refman/
KernSmooth.html’.

### Step 2: Determine Python Equivalents

- **Prioritize Vectorization:** Because R inherently vectorizes operations, you must default to
evaluating ‘numpy’, ‘scipy’, or ‘pandas’ as the primary Python equivalents.
- Choose the library that best matches R’s vectorized nature. For example, if translating R’s ‘exp’,
strongly prefer ‘numpy.exp()’ over the standard library ‘math.exp()’ unless the context
explicitly proves only a scalar is being processed.
```

```

- Carefully map R arguments to Python kwargs (e.g., translating R's 'seq(..., length.out=Q)' to
  Python's 'np.linspace()').

## Output Format Schema

You must output the final conversion guide strictly in well-structured Markdown format. Use the
following outline to ensure consistency and readability:

### 1. Overview of '{language_dependency}' in R

*Provide a concise explanation of what the dependency does, its typical inputs, and expected outputs
.*

### 2. Contextual Usage Analysis

*Summarize how the dependency is applied across the provided dataset. Mention the data types involved
and any recurring patterns found in the source files.*

### 3. Python Conversion Strategy

*State the chosen Python library (e.g., 'numpy') and explain why it is the most robust equivalent for
these specific usage contexts.*

### 4. Step-by-Step Conversion Examples

*For every functional-distinct usage found in the CSV, provide a subsection containing:*
- **Locations:** Files and Function names.
- **Original R Context:** The types of input parameters and return values, and a generalized code
  snippet for demonstration.
- **Python Equivalent:** A complete, executable Python code snippet demonstrating the converted logic
  .
- **Explanation:** A brief breakdown of the syntax translation, addressing any nuances like zero-
  based indexing, argument mapping, or library imports.

```

A.3 Agents Used in Phase 4

convert-r-function-to-python

```

---
name: convert-r-function-to-python
description: Converts an R function to its Python equivalent using a provided JSON dependency map and
  conversion guides.
---

# Convert an R Function to Python

## Description

Rewrite a provided R function into Python. You will receive the R function code, a JSON file
  detailing its dependencies, a folder of language dependency conversion guides, and a target
  output folder. You must maintain the exact logic and functionality of the original code.

## Execution Steps

### Step 1: Infer Types and Define Prototype

Identify all parameters and return types by referencing the R function's CRAN documentation (e.g.,
  the 'KernSmooth' reference manual 'https://cran.r-project.org/web/packages/KernSmooth/refman/KernSmooth.html').
* Create a Python function prototype using complete type annotations.
* **Never omit any types or subtypes inside type annotations.** For example, 'dict[str, int]' or 'np.
  ndarray[Any, np.dtype[np.float64]]' cannot be simplified as 'dict' or 'np.ndarray' if the
  subtypes are known. Also, if there are multiple possible types for a parameter, include all of
  them in the annotation (e.g., 'str | None').
* **Example:** 'def example_function(param1: int, param2: np.ndarray[Any, np.dtype[np.float64]]) -> np.
  ndarray[Any, np.dtype[np.float64]]:'

```

```

### Step 2: Resolve Dependencies

Analyze the provided JSON dependency map. Handle each dependency category as follows:
* **Language Dependencies:** Find the corresponding '{language_dependency}.md' files (e.g., 'min.md',
  'seq.md') in the conversion guides folder for Python translation strategies and examples. You
  don't have to follow them strictly, but they should guide your translation of R constructs to
  Python.
* **Internal Dependencies:** Locate and review the previously converted definitions in the output
  folder ('{function_name}.json'). Strictly follow their Python logic to ensure correct
  integration.
* **External Dependencies:** Map external calls directly to Python's '_KernSmooth' module. *Example:
  Map '.Fortran(F_sdiag)' directly to '_KernSmooth.sdiag()' and can be imported via 'from . import
  _KernSmooth'. The 'F_' prefix is a convention for Fortran calls in R, and the corresponding
  Python function has already been named without this prefix. No functions have return values, and
  the parameters are passed by reference. If you know little about R's Fortran interface, consult
  the documentation 'https://masuday.github.io/fortran_tutorial/r.html'.*

### Step 3: Rewrite Function Body

Translate the R function body into Python syntax.
* Strictly preserve the original logic and functional intent.
* Apply the patterns learned from the language conversion guides.
* Handle R-to-Python structural differences (e.g., 1-based vs. 0-based indexing, data structures,
  control flow) appropriately.

## Output Format

Output strictly valid JSON to '{function_name}.json' in the output folder. **Do not wrap the output
in markdown code blocks.** The function body may contain multiple lines, and you should include
every line as a separate string in the array. Follow the following structure strictly (even if
there are no imports, you must include an empty array for imports):
'''json
{
  "imports": [
    "import numpy as np",
    "from _KernSmooth import sdiag"
  ],
  "function_prototype": "def converted_function(param1: int, param2: np.ndarray[np.float64]) -> np.
    ndarray[np.float64]:",
  "function_body": [
    "    # Function body goes here",
    "    pass"
  ]
}
'''

```

A.4 Supplementary Agent

The following agent is defined in the repository but has not yet been invoked in this project. It is included for completeness and as a reference for the Fortran source analysis task identified as a potential next step in Phase 1.

analyze-c-file-dependencies

```

---
name: analyze-c-file-dependencies
description: Analyzes a provided C file to extract structural dependency information function by
  function (including function-like macros), categorizing calls strictly into internal and
  external dependencies.
---

# Analyze a C File for Dependencies

```

```

## Description

When provided with a C file ('.c' or '.h'), your task is to parse the file, identify every user-defined function or function-like macro, and trace all function-like calls (functions and function-like macros) made within their bodies. You must categorize these dependencies into strict groups and output the result as a structured JSON object.

## Execution Steps

### Step 1: Extract Function or Macro Declarations

Scan the target C file and identify every user-defined function or function-like macro defined within it (e.g., 'int my_func() { ... }'). Create a list of these function or macro identifiers.

### Step 2: Analyze Function or Macro Bodies and Trace Calls

For every function or macro identified in Step 1, thoroughly analyze its execution body. Extract the identifier of every function or function-like macro invoked within that body.

**Exclusions:**
* Completely ignore functions or macros that are part of the C standard library (e.g., 'printf', 'malloc', 'strlen', 'size_t').
* Extract only the identifier (e.g., 'PyArg_ParseTuple', '.Call'), never the arguments or the parentheses.

Categorize the extracted identifiers into the following two mutually exclusive lists. You may need to traverse and scan other files in the same project folder according to the included header files, and you have to use the following heuristics:

* Internal Dependencies:
  * Definition: Identifiers for functions or macros defined within the same C file, or those that appear in other '.c' or '.h' files within the same project folder, which are included via '#include' directives into the target file.
* External Dependencies:
  * Definition: Identifiers for functions or macros belonging to known external, third-party libraries or language bindings (for example, R API calls like 'Rf_protect', Python C API calls like 'Py_Initialize', etc.). Only if you cannot find the definition of the function or macro in the same file or any other file in the same project folder, you should classify it as an external dependency.

### Step 3: Generate JSON Output

Construct a JSON object where each key is the name of a function or macro identified in Step 1. The value for each key must be an object containing exactly two arrays of strings: 'internal_dependencies' and 'external_dependencies'. Deduplicate all identifiers within these arrays.

## Output Format Schema

You must output valid JSON strictly adhering to this structure. Do not include markdown code blocks if the system expects raw JSON.

{
  "my_custom_algorithm": {
    "internal_dependencies": ["helper_function_within_file", "project_utils_init"],
    "external_dependencies": ["Rf_protect", "PyArg_ParseTuple"]
  },
  "helper_function_within_file": {
    "internal_dependencies": [],
    "external_dependencies": []
  }
}

```

B Skill (Command) Specifications

This appendix reproduces the complete specifications of the top-level skills (slash commands) deployed in this project. Each file is stored under `.claude/commands/`. Skills are the entry points invoked interactively; they orchestrate one or more sub-agents (Appendix A) and handle batch dispatch, error recovery, and output persistence.

B.1 Skills Used in Phase 1

analyze-r-folder-dependencies

```

---
name: analyze-r-folder-dependencies
description: Analyzes all R scripts in a folder to extract structural dependency information file by
            file, categorizing function calls into language, internal, and external dependencies.
---

# Analyze an R Folder for Dependencies

## Description

When provided with a target folder, your task is to recursively scan for all R scripts, use the ‘
    @analyze-r-file-dependencies’ agent on each file individually, and save the results as
    structured JSON files.

## Execution Steps

### Step 1: Scan for R Files

Recursively scan the provided target folder and all of its subdirectories for files with ‘.R’ or ‘.r’
    extensions. Compile a complete list of their absolute file paths.

### Step 2: Process and Analyze

Iterate through the list of identified R files sequentially. For each file:
1. Invoke the ‘@analyze-r-file-dependencies’ agent against the file and record its final JSON output.
2. If the command fails or throws an error for a specific file, log the error to the console and
    immediately continue to the next file in your list. Do not halt the entire operation.

### Step 3: Save Output

For every successfully analyzed R file, save the resulting JSON output according to these strict
    rules:
* **Naming Convention:** Use the exact name of the original R script, but replace the ‘.R’ or ‘.r’
    extension with ‘.json’ (e.g., ‘data_cleaning.R’ becomes ‘data_cleaning.json’).
* **Output Location:** If the user did not specify the output folder, save the new ‘.json’ file in ‘
    structural_analysis/{folder_name}/’, where the ‘{folder_name}’ is the name of the target folder.
    The relative path must be the same as its corresponding original R file.

```

B.2 Skills Used in Phase 3

extract-r-folder-language-dependencies

```

---
name: extract-r-folder-language-dependencies
description: Recursively processes a folder of R files, utilizing pre-calculated structural analysis
            JSONs to generate detailed CSV reports of language dependency locations.
---

# Extract Language Dependencies from an R Folder

## Description

```

When provided with a target directory containing R source files and a separate directory containing their corresponding structural analysis results (in JSON format), your task is to process every R script recursively. You will invoke the ‘@extract-r-file-language-dependencies’ agent on each matched file pair (the ‘.R’ file and its corresponding ‘.json’ analysis), and save the extracted dependency data strictly as ‘.csv’ files in a mirrored output directory.

Execution Steps

Step 1: Scan for R Source Files

Recursively scan the provided target directory and all of its subdirectories. Compile a complete list of absolute file paths for every file with an ‘.R’ or ‘.r’ extension.

Step 2: Map to JSON and Process

Iterate through the list of identified R files sequentially. For each R file:

1. **Locate the JSON Input:** Find the corresponding dependency analysis JSON file. It will be located in the provided structural analysis directory, sharing the exact same relative path as the R file, but with the ‘.R’/‘.r’ extension replaced by ‘.json’.
2. **Execute File-Level Agent:** Invoke the ‘@extract-r-file-language-dependencies’ agent, providing it with the R file and its paired JSON file to generate the dependency data.
3. **Error Handling:** If the agent fails, throws an error, or if the required JSON file is missing, log a clear error to the console for that specific file and immediately proceed to the next R file in the list. Do not halt the batch execution.

Step 3: Save Output as CSV

For every successfully analyzed file pair, save the resulting output from the agent according to these strict rules:

- * **Data Format:** The output must be saved strictly as a CSV file.
- * **Naming Convention:** Use the exact base name of the original R script, but replace the ‘.R’ or ‘.r’ extension with ‘.csv’ (e.g., ‘data_cleaning.R’ becomes ‘data_cleaning.csv’).
- * **Output Directory:** Maintain the original directory structure. Save the ‘.csv’ file into the user-specified output directory, using the exact same relative path as the original R file.
- * **Default Output Directory:** If the user did not explicitly specify an output directory, create a default directory path named ‘language_dependency_analysis/{target_folder_name}/’ (where ‘{target_folder_name}’ is the base name of the target R folder) and mirror the relative paths there.

generate-language-dependency-conversion-guides

```
---
name: generate-language-dependency-conversion-guides
description: Orchestrates the generation of conversion guides for translating language dependencies
             from R to Python based on a provided CSV file.
---
```

Generate Language Dependency Conversion Guides

Description

When provided with a target ‘base_folder’ and a CSV file matching the following schema:

```
'''csv
language_dependency,file_path,function_name,line_number,call_body
Re,all.R,bkde,78,"Re(fft(kappa*gcounts, TRUE))"
Re,all.R,bkde2D,156,"Re(fft(rp*sp, inverse = TRUE)/(P1*P2))"
Re,all.R,bkfe,232,"Re(fft(kappam*Gcounts, TRUE))"
abs,all.R,dpill,531,abs(th24Q)
any,all.R,locpoly,610,any(bandwidth <= 0)
as.double,all.R,blkest,265,as.double(x)
as.double,all.R,blkest,265,as.double(y)
as.double,all.R,blkest,267,as.double(xj)
'''
```

Your task is to orchestrate the batch processing of this CSV. You must extract rows corresponding to each unique ‘language_dependency’ into separate CSV-formatted strings (each including the header row), invoke the ‘@generate-language-dependency-conversion-guide’ agent for each subset, and


```

    save the resulting guides into a specified output directory.

## Execution Steps

### Step 1: Identify Unique Dependencies

Parse the input CSV file to identify all unique values in the 'language_dependency' column.

*Note: The CSV is pre-sorted so identical dependencies are grouped together, allowing for efficient sequential processing.*

### Step 2: Extract and Process Each Dependency

Iterate through the list of identified unique 'language_dependency' values sequentially. For each unique dependency:
1. **Extract the CSV Subset:** Isolate all rows matching the current 'language_dependency'. Prepend the standard CSV header row to this subset to create a valid CSV text string.
2. **Execute Conversion Agent:** Invoke the '@generate-language-dependency-conversion-guide' agent. Pass the 'base_folder' and the extracted CSV string as inputs to generate the specific conversion guide.
3. **Error Handling:** If the agent fails or throws an error during generation, log a clear error message to the console specifying the failed 'language_dependency', and immediately proceed to the next dependency in your list. Do not halt the overall batch execution.

### Step 3: Save Output as Markdown Files

For every successfully generated conversion guide, save the agent's markdown output to the file system according to these strict rules:
* **Naming Convention:** Name the output file exactly '{language_dependency}.md' (e.g., 'Re.md', 'abs.md').
* **Output Directory:** Save the files to the user-specified output directory. If the user did not explicitly provide one, create and use a default directory path named 'language_dependency_analysis/conversion_guides/'.

```

B.3 Skills Used in Phase 4

convert-r-file-to-python

```

---
name: convert-r-file-to-python
description: Converts all functions in an R file to Python by parsing dependency graphs and sequentially invoking the function conversion agent.
---

# Convert an R File to Python

## Description

Translate all functions in an entire R file into Python. You will use the R source file, a comprehensive JSON dependency map, a CSV dependency graph, and a language dependency conversion guide folder. You must parse the dependencies, establish the correct conversion order, and invoke the '@convert-r-function-to-python' agent for each function.

## Execution Steps

### Step 1: Extract Function Definitions and Dependencies

* Parse the provided JSON file. The top-level keys represent all the function names within the target R file.
* For each identified function, extract its corresponding R code definition from the R source file.
* Isolate the JSON value associated with each function key to serve as its specific dependency map.

### Step 2: Sort Functions by Dependency Order (Leaves to Roots)

* Filter the provided dependency CSV file by the 'r_file' column to isolate entries relevant only to the current target file.

```

```

* Sort the functions identified in Step 1 topologically, from leaves to roots. Ensure that child
  functions always precede their parent functions in the sorted execution list.
* *Note:* If a child function listed in the CSV is not found in the current R file's JSON/source,
  assume it has been converted elsewhere. Ignore it for the purposes of sorting the current file.

### Step 3: Execute Sequential Conversions

Iterate through the sorted function list from Step 2 sequentially. For each function:
1. **Invoke Conversion Agent:** Call '@convert-r-function-to-python'. Provide the following context:
  * The extracted R code definition.
  * The extracted JSON dependency map.
  * The folder containing language dependency conversion guides.
  * The target output folder for the specific R file. It should be '{target_output_folder}/{
    R_file_name}'. For example, '{target_output_folder}/all.R/'. If the '{target_output_folder}' is
    not specified, the default should be 'conversion_results/R/'.
2. **Error Handling:** If the agent fails or throws an error for a specific function, log a clear
  error message to the console. **Do not halt** the overall batch execution; proceed to the next
  function.

```

combine-python-functions-into-file

```

---
name: combine-python-functions-into-file
description: Combines all converted Python functions (stored as JSON) from a specified directory into
  a unified target Python file, handling import deduplication and project integration.
---

# Combine Converted Python Functions into a File

## Description

Given a source folder containing JSON-formatted function data and a target Python file, your task is
to extract, format, and combine these functions into a single Python script. You must resolve
duplicate imports, format the code syntactically, and integrate it safely into the target
project structure.

## Execution Steps

### Step 1: Discover and Map JSON Files

* Scan the specified source folder for all files ending with '.json'.
* The filename (excluding the '.json' extension) represents the original R function name.

### Step 2: Extract and Format Function Definitions

For each JSON file, extract the 'imports', 'function_prototype', and 'function_body'.
* **Concatenation:** Combine the 'function_prototype' and the 'function_body' array into a single
  multi-line string.
* **Indentation:** Ensure standard Python indentation. The 'function_prototype' should have zero
  indentation, and every line within the 'function_body' must be indented by exactly 4 spaces
  relative to the prototype.

*Example JSON Structure:*
'''json
{
  "imports": [
    "import numpy as np",
    "from KernSmooth import _KernSmooth"
  ],
  "function_prototype": "def rlbins(X: np.ndarray[np.float64], Y: np.ndarray[np.float64], gpoints: np.
    ndarray[np.float64], truncate: bool = True) -> dict:",
  "function_body": [
    "    n = len(X)",
    "    M = len(gpoints)",
    "    return {\"xcounts\": xcnts, \"ycounts\": ycnts}"
  ]
}
'''

```

```
'''
### Step 3: Deduplicate and Sort Imports

* Aggregate all 'imports' arrays from every JSON file.
* Remove any duplicate import statements.
* Sort the deduplicated imports logically:
  1. Standard library imports (e.g., 'import math').
  2. Third-party imports (e.g., 'import numpy as np').
  3. Local project imports (e.g., 'from .linbin import linbin').
* Combine the sorted imports and the formatted function definitions into one final multi-line string.

### Step 4: Validate and Output the Combined Program

Do not blindly overwrite the target Python file. You must ensure the generated code aligns with the
existing project architecture:
1. Structural Scan: Analyze the target project's build and configuration files (e.g., 'pyproject.
  toml', 'meson.build', '__init__.py') to verify correct module naming conventions and dependency
  paths.
2. Binding Corrections: If the structural scan reveals discrepancies in how external C/Fortran
  bindings are exposed (e.g., '_KernSmooth' is built under a different path than what the JSON
  imports suggest), automatically fix the import paths in your combined string to match the
  reality of the build system.
3. Safe Write: Output the finalized string into the target Python file. If the file already
  contains code, integrate the new functions safely without breaking existing syntax or
  overwriting required project-level configurations.
```

B.4 Workflow Utility

The following skill was used at the end of each phase to produce the structured phase summaries stored in `docs/daily_summaries/`, which are the primary source material for this report.

summarize-research-report

```
---
name: summarize-research-report
description: Summarizes the current session's activities, modifications, and findings into a precise,
  formal, and concise research report.
---

# Summarize Session into Research Report

This command analyzes the current working session and synthesizes the activities into a formal,
concise research report.

## Prompt

Review the history of the current session, including all user requests, tool executions, codebase
queries, and file modifications. Based on this context, generate a formal research report
summarizing the work accomplished.

The report must adhere strictly to the following parameters:
- Tone: Precise, formal, and objective. Avoid conversational filler.
- Length: Highly detailed in facts but strictly concise in wording. Do not exceed 500-700 words
  unless absolutely necessary to capture critical technical details.
- Specificity: Explicitly mention specific file names, library versions, metrics, or core
  architectural decisions discussed or modified during the session. Avoid generic summaries.

Please structure the report using the following markdown format:

### 1. Abstract
```

Provide a 2-3 sentence high-level overview of the session's primary objective and the final outcome or state of the project.

2. Methodology & Actions Taken

Detail the concrete steps taken during the session. Include:

- Which files were created, modified, or analyzed.
- What tools or commands were executed.
- Any bugs investigated and the approach taken to resolve them.

3. Key Findings & Results

Highlight the core results of the session:

- Output metrics, successful compilations, or structural changes.
- Key technical insights or discoveries made during the work.
- Resolutions to any problems encountered.

4. Conclusion & Next Steps

Briefly conclude the report by stating the final status of the session's goal.